

TFACC2 Linux 驱动源码导读

对应驱动版本： 0.7.1

主要源码： [tfacc2.c](#)、[tfacc2.h](#)、
[tfacc_hugepage_uapi.h](#)、
[tfacc_cache_uapi.h](#)、
[tfacc_address_uapi.h](#)、
[tf_hugepage_register.c](#)

这份文档面向第一次接触 Linux 驱动的读者。目标不是只描述“代码做了什么”，还要解释用户态、内核态、NPU 硬件和 TFEngine 之间为什么要这样连接。

文档覆盖以下范围：

- `tfacc2.c` 中当前所有有效函数；
- `tf_hugepage_register.c` 中所有函数；
- `tfacc2.h`、三个 UAPI 头文件、Makefile 和安装脚本；
- TFEngine 中与 64 位地址、高位寄存器、成对锁和 cache invalid 相关的桥接代码；
- 当前实现的已知风险和后续重构建议。

寄存器中部分 magic number 的准确硬件语义必须以 NPU40T 寄存器手册为准。

本文对这些值的描述只采用源码能够确认的用途，不猜测未公开的 bit 定义。

阅读导航

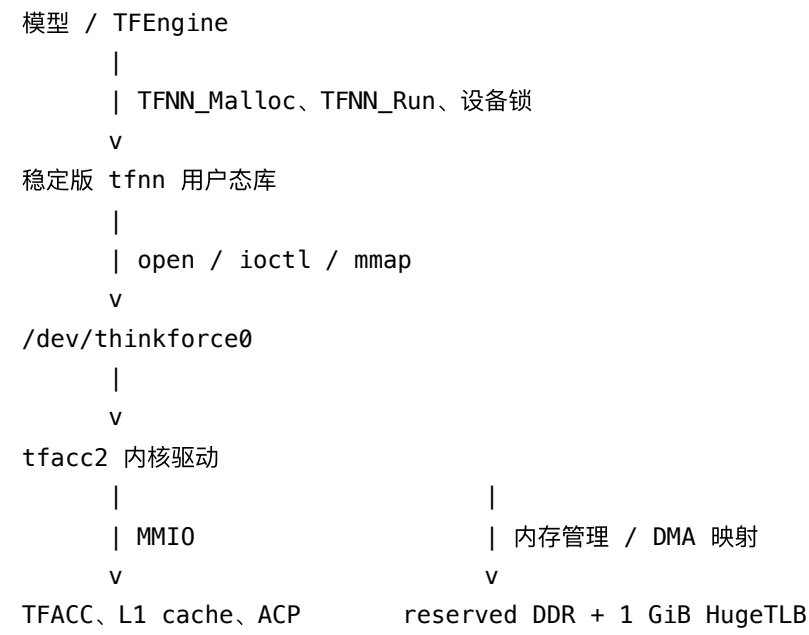
- 第 1~3 章：先理解驱动在整个软件栈中的位置和 Linux 基础概念；
- 第 4~6 章：认识数据结构、ioctl 协议以及模块加载/卸载；
- 第 7~12 章：理解 reserved DDR、HugePage、mmap、cache invalid、高位寄存器和锁；
- 第 13 章：逐个阅读 `tfacc2.c` 中的函数；
- 第 14~16 章：阅读 HugePage 工具、安装脚本和 TFEngine 调用侧；
- 第 17~19 章：学习调试方法、识别历史风险并按建议顺序实践。

如果是第一次看 Linux 驱动，建议先读第 1、2、5、6、7、9 章，再进入第 13 章。

遇到函数名时可以直接在编辑器里搜索；文档中的分组顺序基本对应源码职责，而不是机械照搬函数出现顺序。

1. 先建立整体认识

1.1 驱动在系统中的位置



驱动主要承担六件事：

1. 发现并初始化 NPU40T 硬件；
2. 把 TFACC 和 cache 寄存器映射到内核；
3. 创建 /dev/thinkforce0，提供 open/ioctl/mmap/release；
4. 从 reserved DDR 或注册的 HugePage 中给 tfnn 分配 DMA 地址；
5. 管理跨进程 TFACC 锁、cache invalid 和共享 L1 高位地址；
6. 在卸载或进程退出时回收软件状态和硬件状态。

1.2 最常见的一条执行路径

tfnn 打开 /dev/thinkforce0

-> tf_open()

tfnn 请求一大块内存

-> ioctl(TF_BUF_CREATE)

-> tf_ioctl()

-> tf_ioctl_create()

-> tf_create_and_init_kbuf()

-> tf_init_kbuf()

-> 优先 reserved DDR, 耗尽后使用 HugePage

-> 返回 dma_addr + mmap_id

tfnn 映射这块内存

-> mmap(fd, offset = mmap_id << PAGE_SHIFT)

-> tf_mmap()

-> remap_pfn_range()

TFEngine 构建算子

-> 检查输入输出处于同一 4 GiB 窗口

-> 把 address_high 保存为命令缓冲区的软件元数据

TFEngine 执行算子

-> 跨进程锁住共享 L1 的两个 TFACC

-> ioctl(TF_SET_ADDRESS_HIGH)

-> tf_ioctl_set_address_high()

-> CPU MMIO 修改 cache 侧 0x90 / 0x94

-> tfnn 启动 NPU 命令

2. 阅读驱动前必须知道的 Linux 概念

2.1 用户态和内核态

普通应用不能直接解引用内核指针，也不能直接信任用户传入的地址。因此 ioctl 处理函数必须使用：

- copy_from_user()：把用户结构体复制进内核；
- copy_to_user()：把结果复制回用户空间；
- __get_user() / __put_user()：读写一个简单用户值。

失败通常返回 -EFAULT。内核函数返回负 errno，用户态看到的是 -1，同时 errno 被设置成对应正数。

2.2 字符设备

`/dev/thinkforce0` 是字符设备节点。驱动通过 `struct file_operations` 注册四个入口：

用户操作	内核回调
<code>open()</code>	<code>tf_open()</code>
<code>close()</code>	<code>tf_release()</code>
<code>mmap()</code>	<code>tf_mmap()</code>
<code>ioctl()</code>	<code>tf_ioctl()</code>

`filp->private_data` 用来把一次文件打开关联到 `struct tf_device`。

2.3 MMIO、`ioremap()`、`readl()`、`writel()`

TFACC 寄存器是物理地址空间中的设备寄存器，不是普通 RAM。内核先通过 `ioremap(physical, length)` 得到 `__iomem` 地址，再用 `readl/writel` 访问。

新代码访问高位寄存器时使用 `readl/writel`，原因是它们表达了 MMIO 语义，并包含体系结构需要的访问约束。源码中的许多历史函数仍使用 `volatile unsigned int *` 直接读写，能在目标平台运行，但不建议作为新代码模板。

2.4 四种容易混淆的地址

名称	本项目字段	含义
用户虚拟地址	HugePage 工具的 <code>user_addr</code>	用户进程看到的地址，只在该进程页表中有意义
CPU 物理地址	<code>physical_addr</code> / <code>cpu_phys_addr</code>	CPU 页表和 <code>remap_pfn_range()</code> 使用的物理地址
DMA 地址	<code>dma_addr</code> / <code>kbuf.phy_addr</code>	NPU 在总线上看到的地址，开启 IOMMU 时可能不同于物理地址
内核虚拟地址	<code>ioremap()</code> 返回值	内核访问设备 MMIO 时使用

`kbuf.phy_addr` 这个名字是历史命名；对 HugePage 后端来说它实际保存的是 DMA 地址。`kbuf.cpu_phys_addr` 才是映射给 CPU 用户进程时需要的物理地址。

2.5 PFN、页和 `remap_pfn_range()`

PFN 是 Page Frame Number，即物理地址除以 `PAGE_SIZE`。`tf_mmap()` 使用：

```
remap_pfn_range(vma, vma->vm_start,
                cpu_phys_addr >> PAGE_SHIFT,
                size, vma->vm_page_prot);
```

把一段物理页直接装进调用进程的页表。用户随后读写返回的虚拟地址，实际访问的就是 reserved DDR 或 HugePage 对应物理内存。

2.6 PID 和 TGID

Linux 中每个线程有独立 PID，同一进程所有线程共享 TGID。这个驱动把资源所有权放在 TGID 上，因为一个线程锁设备、另一个线程执行或解锁是合法场景。

2.7 mutex 和 spinlock

锁	能否睡眠	本驱动用途
mutex	可以	内存池、链表、跨进程设备锁、PID 表
spinlock	不可以	很短的 isBusy 更新

持有 spinlock 时不能调用可能睡眠的函数。本文后面的“已知问题”会指出当前代码中仍存在的历史 spinlock 使用风险。

2.8 内核常见的 goto 清理结构

内核 C 经常这样组织错误回滚：

```
申请 A
申请 B 失败 -> goto free_a
申请 C 失败 -> goto free_b
成功返回

free_b: 释放 B
free_a: 释放 A
return error
```

它不是随意跳转，而是在没有 C++ RAII 的情况下保证每条失败路径只释放已经成功获取的资源。tf_ioctl_register_hugepage() 和 tf_ioctl_create() 都采用这种形式。

3. 文件地图

文件	功能
tfacc2.h	内核头文件、寄存器地址、内部结构体、file operations、ACPI/模块入口
tfacc2.c	驱动主体：初始化、MMIO、内存、mmap、ioctl、锁和卸载
tfacc_hugepage_uapi.h	HugePage 注册/查询/清理 ABI
tfacc_cache_uapi.h	64 位 cache invalid ABI
tfacc_address_uapi.h	共享 L1 高位地址 ABI
tf_hugepage_register.c	用户态 HugePage 管理工具
Makefile	Kbuild 模块和用户态工具构建规则
build_driver.sh	制作临时构建目录并安装模块
../../insmodTFDriver.sh	加载、卸载驱动并安装用户工具

4. tfacc2.h：驱动的静态定义

4.1 模块元数据和调试宏

- DRIVER_VERSION 是模块版本，当前为 0.7.1；
- MODULE_AUTHOR/DESCRIPTION/LICENSE/VERSION 会进入 .ko 元数据；
- DPRINTK 仅在定义 TF_DEBUG 时输出；
- assert 和 DASSERT2 也是调试日志，不会像用户态 assert() 那样终止执行。

查看已加载版本：

```
cat /sys/module/tfacc2/version
```

4.2 寄存器地址和 mmap ID

- TFACC0_BASE ... TFACCLITE3_BASE：八个 TFACC 寄存器窗口；
- TFACC*_CACHE_BASE：八个 cache 寄存器窗口；
- TFACC*_FULL_ACP_BASE：四个共享 L1/ACP 控制块，每两个 TFACC 一个；
- chipGap：多 chip 时同类寄存器窗口之间的物理地址差；
- REG2ID/CACHEREGID/...：复用 mmap 的 page offset 来选择映射对象。

4.3 enum tf_memory_backend

标记一个驱动大块来自哪里：

- TF_MEMORY_RESERVED：固件/ACPI/DT 预留 DDR；
- TF_MEMORY_HUGEPAGE：用户工具注册后被驱动长期 pin 的 1 GiB HugeTLB 页。

4.4 struct kbuf

一个 kbuf 不是每个 tensor，而是驱动分给 tfnn 的一个大块。tfnn 会在用户态继续把它切成 16 KiB 小页。

重要字段：

字段	作用
list	放进 dev->buf_list 哈希表
cpu_phys_addr	mmap 给 CPU 时使用
phy_addr	返回给 NPU/tfnn 的 DMA 地址
len	大块字节数
mmap_id	用户随后传给 mmap 的选择 ID
owner_tgid	所有进程线程组
backend	reserved 或 HugePage
huge_region	HugePage 后端所属区域
pool_offset	在内存池中的起始偏移
pool_previous_offset	出错时回滚 HugePage bump pointer
pool_index	reserved block 下标；HugePage 为 -1

4.5 struct tf_device

它代表 /dev/thinkforce0 对应的驱动实例：

- ioreg[]：所有 TFACC MMIO 的内核映射；
- ioreg_cache[]：所有 cache MMIO 的内核映射；
- reg_buf[]/cache_reg_buf[]：这些映射的物理地址、长度和 mmap ID；
- major/minor/cdev/device：字符设备对象；
- dma_device：DMA API 使用的 platform device；
- isBusy：当前打开的文件描述符数量；
- buf_list：已分配驱动大块的哈希表；
- mmap_id_counter：动态大块 ID 生成器；

- useDDR2：当前申请选择的 chip/NUMA 编号，名称是历史遗留；
- holdTFACCPid[]/holdTFACCTgid[]：跨进程 TFACC 锁所有者。

4.6 ioctl 传输结构体

- tf_buf_io_param：旧版内存申请 ABI，输入长度/chip/uncache，输出 DMA 地址和 mmap ID；
- tf_version：SDK/内核版本握手；
- tf_app_info：记录打开驱动的 PID/TGID；
- tf_lock_record：TFACC 锁的历史记录；
- ReserveDDRBlock：256 MiB reserved block 的起点、长度、偏移和所有者。

4.7 tf_device_ops

把系统调用与驱动函数绑定：

```
static const struct file_operations tf_device_ops = {
    .owner          = THIS_MODULE,
    .open           = tf_open,
    .release        = tf_release,
    .mmap           = tf_mmap,
    .unlocked_ioctl = tf_ioctl,
};
```

.owner = THIS_MODULE 防止文件仍被使用时模块被安全地卸载。

4.8 ACPI 与非 ACPI 入口

ACPI 构建通过 module_platform_driver(tf_tfacc_driver) 注册 platform driver，匹配 ID TFA0001 后调用 tf_init_module(pdev)。非 ACPI 构建使用传统 module_init/module_exit。

HugePage DMA 注册依赖 pdev->dev，因此当前仅 ACPI 路径启用 HugePage 扩展。

5. UAPI：用户态和内核态共同遵守的协议

UAPI 头文件同时被工具/Engine 和驱动理解。字段使用 __u32/__u64/__s32，是为了固定 ABI 位宽。

5.1 ioctl 编号表

ioctl	NR	方向	功能
TF_BUF_CLEAR	0	无参数	清空所有驱动大块状态
TF_BUF_CREATE	0	双向	申请一个驱动大块
TF_BUF_FLUSH1	1	双向	头文件保留，当前 dispatcher 没有实现
TF_VERSION_CHECK	2	双向	SDK 版本检查
TF_APP_LOCK	3	写	获取一个 TFACC 的跨进程锁
TF_APP_UNLOCK	4	写	释放 TFACC 锁
TF_READ_PIDS	5	读	获取打开驱动的 PID
TF_READ_APP_LOCK_RECORD	6	读	获取锁历史
TF_READ_RESERVE_MEM_RECORD	7	读	查看 reserved block
TF_BUF_RESET	8	无参数	硬件复位并重新初始化
TF_READ_APP_USAGE	8	读	获取使用率统计
TF_HUGEPAGE_REGISTER	9	双向	注册一页 1 GiB HugeTLB
TF_HUGEPAGE_QUERY	10	双向	按下标查询注册页
TF_HUGEPAGE_CLEAR	11	双向	清除未使用的注册页
TF_CACHE_INVALIDATE	12	写	对完整 64 位 DMA 范围做 cache invalid
TF_SET_ADDRESS_HIGH	13	写	设置某个 TFACC pair 的 L1 地址高位

相同 NR 不一定冲突，因为 ioctl 命令值还编码方向和参数大小，switch 比较的是完整命令值。TF_MAX_NR = 14 表示允许的最大 NR 是 13。

5.2 HugePage UAPI

tf_hugepage_register 输入用户映射、长度和 chip，输出 CPU 物理地址、DMA 地址和区域 ID。当前只接受恰好一页 1 GiB。

tf_hugepage_info 用于 --list，其中：

- allocated 是驱动已经 bump 分配出去的字节数；
- owner_tgid = -1 表示尚未绑定进程；
- physical_addr 和 dma_addr 不保证相同。

5.3 cache invalid UAPI

`tf_cache_invalidate` 带完整 `dma_addr + length + chip_id`。驱动会验证这个范围确实属于当前 TGID，避免任意进程用 `ioctl` 干扰别人的 cache。

5.4 高位地址 UAPI

`tf_address_high` 带 `tfacc_id` 和 `address_high`。硬件字段只有 8 位，所以这里的有效范围是 `0x00..0xff`，对应总线地址 bits [39:32]，完整可寻址空间为 40 位。

调用者必须同时持有该 pair 的两个 `TF_APP_LOCK`，例如设置 TFACC 0 的高位前，当前 TGID 必须同时持有 0 和 1。

所有新 UAPI 都带 `api_version` 和保留为 0 的 `flags`。这样未来扩展结构体语义时可以显式拒绝不兼容请求，而不是悄悄误解字段。

6. 驱动加载和卸载全过程

6.1 `tf_init_module()`

这是最重要的初始化入口。

按执行顺序：

1. ACPI 路径通过 `platform_get_resource()` 取得 reserved DDR 起点和长度；
2. 非 ACPI 路径从 DT `/reserved-memory/buffer@1` 读取起点；
3. `readSocketInfo()` 判断 chip 数、chipGap，并把 reserved DDR 切成 256 MiB block；
4. 注册一个历史遗留的 `thinkforce_kernel` 主设备号；
5. 初始化 PID、app lock 和统计锁；
6. 读取 ACP 初始化标志，决定是否跳过 full cache 软件复位；
7. 对每个 chip 打开 full/lite 时钟；
8. 调用 `tfacc_full_acp()` 设置 direct 和 cache 初始高位；
9. 初始化各 TFACC cache；
10. 打印每个 TFACC 的版本和 ID；
11. 创建 `thinkforce_class`；
12. `tf_create_and_init_device()` 映射所有 TFACC/cache MMIO；
13. ACPI 路径设置 64 位 DMA mask；
14. `tf_create_and_init_cdev()` 创建 `/dev/thinkforce0`。

为什么 reserved DDR 必须最先发现：驱动初始化 ACP direct 高位、构造内存池和稳定版 `tfnn` 的结果队列都依赖它。

6.2 tf_cleanup_module()

卸载入口：

1. tf_remove() 删除字符设备、软件 buffer 和 HugePage 注册；
2. 销毁 device class 和历史字符设备号；
3. 对每个 chip 关闭 full/lite 时钟；
4. tfacc_full_checkrstcond() 更新复位状态标记。

6.3 tf_remove()

负责驱动对象层面的拆除：

- 解除 device 的 drvdata；
- 删除 /dev/thinkforce0 对应 cdev；
- 删除 kbuf 元数据并重置 reserved/HugePage 分配游标；
- DMA unmap、unpin 并释放全部 HugePage region；
- 释放 tf_device 。

7. reserved DDR 与 HugePage 混合内存模型

7.1 为什么不直接用普通 malloc

NPU 需要稳定的总线地址。普通用户页可能不连续、会回收或迁移，也未必有适合设备的 DMA 映射。驱动使用两类可控内存：

- 固件预留、物理连续的 reserved DDR；
- 用户申请后由驱动 FOLL_LONGTERM pin 并通过 DMA API 映射的 1 GiB HugeTLB 页。

7.2 reserved DDR 如何切分

readSocketInfo() 把每个 chip 的 reserved DDR 切成最多若干个 256 MiB

ReserveDDRBlock 。每块是简单 bump allocator：

```
startPos
|
+----- 已分配 offset -----+----- 剩余空间 -----+
```

一个 block 同时只属于一个 TGID。驱动没有逐小块 free；通常在进程关闭设备时把整个 TGID 的 block offset 清零。

7.3 为什么第一块必须来自 reserved DDR

稳定版 tfnn 第一次向驱动申请的内存中放置结果队列，但它把所有 REG_RESULT_QUEUE*_HIGH 写成 0。硬件依靠 ACP 0x10 中驱动启动时设置的 reserved DDR 高位补全地址。

因此 tf_init_kbuf() 要求每个 TGID/chip 先拥有至少一个 reserved block，称为 reserved anchor。没有 anchor 时不能直接从 HugePage 返回第一块，否则结果队列会写到错误位置。

7.4 HugePage fallback

reserved DDR 找不到足够空间后，tf_init_kbuf() 遍历 tf_huge_regions：

1. chip 必须匹配；
2. region 必须空闲或已属于当前 TGID；
3. 从 region->offset 开始寻找候选范围；
4. DMA 低 32 位恰好为 0 时跳过 1 MiB；
5. 单次驱动大块不能跨越 4 GiB 高位边界；
6. 成功后记录 DMA 地址、CPU 物理地址、region 和旧 offset。

不允许单个大块跨 4 GiB 边界，是因为 NPU 算子中的部分地址字段只有低 32 位，一次算子只能配合一个共享 address_high。

7.5 分配不是逐 tensor 发生的

TF_BUF_CREATE 分的是 tfnn 大块。tfnn 再把大块切成自己的 16 KiB 页面并管理 tensor。驱动因此只知道“大块属于哪个 TGID”，不知道每个 tensor 的生命周期。

8. HugePage 注册全过程

8.1 用户工具侧

检查/扩容 1 GiB HugeTLB 池

- > memfd_create(MFD_HUGETLB | MFD_HUGE_1GB)
- > ftruncate 到请求容量
- > 每次 mmap 一页 1 GiB
- > 可选 mbind 到 NUMA node
- > ioctl(TF_HUGEPAGE_REGISTER)
- > 驱动 pin 页面并 DMA map
- > 工具 munmap；内核 pin 仍保持页面有效

8.2 tf_ioctl_register_hugepage() 为什么步骤很多

它必须防止用户把普通匿名页或不连续页伪装成设备内存：

1. 要求 CAP_SYS_ADMIN ；
2. 检查 UAPI version、flags 和 chip；
3. tf_validate_hugetlb_mapping() 确认 VMA 是对齐的 1 GiB HugeTLB；
4. pin_user_pages_fast(... FOLL_LONGTERM) 长期固定所有 base page；
5. 逐 PFN 验证物理连续；
6. 建立单项 scatterlist；
7. dma_map_sg() 得到设备 DMA 地址；
8. 检查没有和已注册物理范围重叠；
9. 加入全局 region 链表；
10. 把物理地址、DMA 地址和 region ID 返回用户。

工具在 ioctl 后 munmap() 没有问题，因为驱动持有 page pin。真正释放发生在 TF_HUGEPAGE_CLEAR 或驱动卸载时。

8.3 为什么物理地址和 DMA 地址都要保留

- NPU 命令使用 DMA 地址；
- 用户态 mmap 建页表要用 CPU 物理地址；
- 无 IOMMU 时二者通常相同；
- 有 IOMMU 时必须严格区分。

9. mmap 如何复用 offset 选择对象

用户传给 mmap 的 offset 必须页对齐。tfnn 把 mmap_id 左移 PAGE_SHIFT 作为 offset，内核在 vma->vm_pgoff 中收到原始 ID。

tf_mmap() 根据 ID 分派：

ID	映射内容
0	TFACC 0 寄存器
REG2ID...	其他 TFACC 寄存器
CACHEREGID...	cache 寄存器
SRAM1ID/SRAM2ID/REGMAINID	历史保留对象
动态 mmap_id	reserved/HugePage 驱动大块

寄存器映射使用 `pgprot_noncached()`，避免 CPU cache 把 MMIO 读写合并成普通内存行为。申请带 `uncache` 标记的数据块时，下一次数据 `mmap` 使用 `pgprot_writecombine()`。

`VM_LOCKED` | `VM_DONTEXPAND` | `VM_DONTDUMP` 用来避免映射被扩展、换出或进入 core dump。

10. cache invalid 为什么需要新 ioctl

旧 `TF_TFNN_InvalidateCache()` 把 invalid 地址高 32 位固定为 0。高地址 HugePage 出现后，它可能 invalid 错误的地址。

`TF_CACHE_INVALIDATE` 的路径是：

```
TFEngine DriverInvalidateCache(full_dma_addr)
-> ioctl
-> tf_ioctl_cache_invalidate()
-> 验证范围属于当前 TGID
-> tf_cache_invalidate_dma_range()
-> 对 chip 的 8 个 cache 写 low/high/length/request
-> 轮询 ACK, 最多 1 秒
```

驱动在返回新的高地址大块前先 invalid 一次；TFEngine 在释放高地址 `MmapBuf` 前再 invalid 一次。这样不需要修改稳定版 tfnn 的 ABI。

11. 共享 L1 高位地址：最关键的硬件逻辑

11.1 为什么每两个 TFACC 只有一个状态

硬件结构是：

```
TFACC 0 ----+
             +---- shared L1 / ACP high state 01
TFACC 1 ----+

TFACC 2 ----+
             +---- shared L1 / ACP high state 23
TFACC 3 ----+
```

Lite TFACC 45、67 同理。因此一个 TFACC 改高位时会影响同 pair 的另一个 TFACC。

11.2 运行时为什么只改 0x90/0x94

ACP 中存在两类不同路径：

- 0x10：direct TFACC 路径高位；
- 0x90/0x94：共享 L1 cache 操作数路径高位。

驱动初始化时二者都指向 reserved DDR。运行时只允许 cache 操作数在不同 4 GiB 窗口之间切换。

稳定版 tfnn 的结果队列 HIGH 寄存器始终写 0，它依赖 0x10 保持 reserved DDR 高位。如果运行时把 0x10 也改成模型 tensor 的高位，NPU 完成信息就会写到错误地址，软件永远等不到 result tail，表现为所有 NPU 操作卡死。这就是 0.7.0 出错、0.7.1 修复的原因。

11.3 为什么必须“掩码替换”而不是 |=

假设旧高位是 0x09，新高位是 0x08：

```
0x09 | 0x08 = 0x09    // 错，bit 0 没被清掉
```

正确做法：

```
value &= ~HIGH_MASK;
value |= new_high;
```

因此 tfacc_write_cache_address_high() 先清掉 bits [7:0] 和 [23:16]，再写入新值，同时保留其他控制位。

11.4 为什么需要 wmb() 和读回

CPU 对设备写可能经过总线缓冲。wmb() 保证前面的 MMIO 写在后续命令启动前按序可见；随后 readl(0x94) 作为 posted-write readback，迫使写事务到达设备。

11.5 为什么驱动还要校验两个锁

TFEngine 的 C++ mutex 只能约束同一个进程。另一个进程有自己独立的 mutex，不能看到本进程状态。

跨进程隔离由驱动的 TF_APP_LOCK 完成。高位 ioctl 在 app_mutex 下检查 pair 两颗 TFACC 的 holdTFACCTgid 都等于 current->tgid，然后才做 MMIO。检查和写入与 TF_APP_UNLOCK 互斥，避免“刚检查完，另一个线程解锁，别的进程立刻抢走”的时间窗口。

12. 跨进程锁和利用率统计

12.1 锁状态

`holdTFACCPid[id]` 保存最初加锁线程，`holdTFACCTgid[id]` 保存进程。所有权判断以 TGID 为准，所以同一进程的其他线程可以继续使用或解锁。

12.2 TF_APP_LOCK 参数

用户传两个 int：

```
params[0] = 等待策略/微秒数  
params[1] = tfacc_id
```

- `< 0`：尝试一次；
- `> 0`：每 10 微秒重试，直到预算耗尽；
- `== 0`：注释说永久等待，但当前实现实际返回失败。

12.3 为什么高位方案要成对锁

如果进程 A 运行 TFACC 0，同时进程 B 运行 TFACC 1，二者可能各自写不同的共享 L1 高位。成对锁住 0、1 后，一个 pair 同时只能属于一个进程；进程内部再允许相同高位的两个算子并行。

12.4 使用率统计

`tf_use_record` 保存最多约 300 秒的一秒粒度环形队列，并保留当前未封口时间片。加锁/解锁时调用 `updateUseRecord()`，查询时计算 15/60/300 秒利用率。

13. tfacc2.c 全部函数说明

下面按源码分组列出所有函数。对关键函数给出更细的步骤；简单硬件 helper 则说明其职责和调用原因。

13.1 低层复位和 HugePage 公共 helper

`cpu_write(base, value)`

- 临时 `ioremap` 4 字节物理寄存器；
- 写一个 32 位值后立刻 `iounmap`；
- 只被 `hardware_tfacc_reset()` 使用。

它是最小 MMIO helper。当前没有检查 `ioremap` 失败，也使用普通 `volatile` 写法；新代码应优先使用 `writel()` 并检查 `NULL`。

hardware_tfacc_reset()

- 按固定顺序写 full TFACC 时钟、cache 复位和 reset 控制寄存器；
- `efuse` 模式直接返回；
- 用于 `TF_BUF_RESET` `ioctl` 的完整硬件恢复路径。

顺序不能随意调整，因为通常包含“clamp -> cache reset -> assert reset -> deassert reset -> unclamp”。准确含义需对照硬件手册。

tf_ranges_overlap(a, a_len, b, b_len)

判断两个半开区间 `[a,a+a_len)` 与 `[b,b+b_len)` 是否相交。注册 HugePage 时用它防止同一物理内存被重复注册和重复 DMA map。

tf_unpin_huge_region(region)

按获取资源的反方向释放：

1. 如果 DMA map 成功，先 `dma_unmap_sg()`；
2. `unpin_user_pages_dirty_lock(..., true)` 解除长期 pin 并按可能被设备写过处理 dirty；
3. 释放 page 指针数组和 region 结构体。

tf_release_all_huge_regions()

持有 `tf_memory_mutex`，安全遍历全局 HugePage 链表，摘链并调用 `tf_unpin_huge_region()`。驱动卸载时使用。

tf_validate_hugetlb_mapping(start, length)

- 要求调用进程有 `mm`；
- 长度必须正好 1 GiB，地址也必须 1 GiB 对齐；
- 在 `mmap_read_lock()` 下查 VMA；
- 验证整个范围属于同一 VMA、VMA 是 hugetlb、内核页大小是 1 GiB。

它拒绝 Transparent HugePage，因为 THP 不保证这里需要的显式 1 GiB HugeTLB 语义。

13.2 HugePage ioctl 函数

tf_ioctl_register_hugepage(dev, arg)

完整流程见第 8 章。关键设计点：

- `CAP_SYS_ADMIN` 限制长期 pin 大量系统内存的能力；
- `FOLL_LONGTERM` 明确告诉内核这是设备长期 DMA 使用；

- PFN 连续性检查确保单页可以用一条 SG 描述；
- DMA map 后使用 `sg_dma_address()`，不能假设等于 `page_to_phys()`；
- `copy_to_user` 失败时必须从链表移除并逐层回滚。

tf_ioctl_query_hugepage(arg)

按用户给出的零基 `index` 遍历 `region` 链表，返回地址、容量、已分配偏移、所有者、`chip` 和 ID。越过末尾返回 `-ENOENT`，用户工具用它作为列表结束标记。

tf_ioctl_clear_hugepages(dev, arg)

- 要求 `CAP_SYS_ADMIN`；
- `dev->isBusy` 必须等于 1，表示只有执行 `--clear` 的工具自身打开设备；
- 所有 `region` 必须 `owner_tgid == -1`；
- 然后逐页 DMA unmap 和 unpin。

这两个检查是为了避免 NPU 应用仍在物理页时把它归还给 Linux。

13.3 cache invalid 函数

tf_cache_invalidate_dma_range(dev, chip_id, dma_addr, length)

- 验证 `chip`、长度和整数溢出；
- 遍历该 `chip` 的 8 个 `cache`；
- 写 `invalid` 地址低位、高位、长度和 `request`；
- 每个 `cache` 最多轮询 ACK 1 秒；
- 超时打印 `cache/core/DMA` 范围并返回 `-ETIMEDOUT`。

为什么所有 8 个 `cache` 都做：一块内存可能曾被同 `chip` 的不同 TFACC 使用，驱动无法只凭 `buffer` 元数据知道具体驻留在哪一个 `cache`。

tf_kbuf_chip_id(kbuf_p)

根据 backend 找出 `buffer` 所属 `chip`：HugePage 从 `region` 取，`reserved` 从 `block` 取。

tf_dma_range_owned_locked(dev, tgid, chip_id, dma_addr, length)

在 `dev->buf_list` 中查找一个同 `TGID`、同 `chip`、且完整包含请求范围的驱动大块。

函数名中的 `_locked` 表示调用者必须已经持有 `tf_memory_mutex`。

tf_ioctl_cache_invalidate(dev, arg)

复制并验证 UAPI 请求，在 `tf_memory_mutex` 下完成所有权检查，然后调用底层 `invalid`。不允许进程 `invalid` 不属于自己的 DMA 地址。

tf_get_reserve_ddr_blocks(p)

把全部 `ReserveDDRBlock` 顺序复制给用户，用于管理工具观察 `reserved DDR` 分配状态。

13.4 app lock 和使用率函数

initRecords()

清空 500 条锁历史、每个 TFACC 对应的历史下标、使用率环形队列和当前位置。

updateUseRecord(tfaccID, lastHolding, curMs)

这是统计核心：

1. 丢弃超过 300 秒的已封口桶；
2. 根据上一状态是否持锁，把经过的完整秒写入环形队列；
3. 更新当前秒尚未封口的占用毫秒；
4. 把状态切换为 lastHolding 。

参数名 lastHolding 容易误解，它实际表示“本次更新后的 holding 状态”。

insertAppLockRecord(dev, tfaccID)

记录当前 PID/TGID、加锁时间和 TFACC ID；更新设备所有者数组；推进 500 项环形历史；并通知使用率统计进入 holding 状态。调用者持有 app_mutex 。

finishAppLockRecord(dev, tfaccID)

把设备所有者重置为 -1、记录解锁时间、关闭 holding 标志，并通知使用率统计。调用者持有 app_mutex 。

tf_app_try_lock(dev, p)

- 从用户数组读取等待时间和 TFACC ID；
- 同 TGID 已持有则幂等成功；
- 在 app_mutex 下检查空闲并写入所有者；
- 未成功时按等待参数重试。

当前通过 udelay(10) 忙等，会占用 CPU；更标准的实现可改成 waitqueue/completion。

tf_app_try_unlock(dev, p)

读取 TFACC ID；只有当前 TGID 是所有者时才真正释放。未持锁也返回成功，保持旧 ABI 的幂等行为。

tf_app_release_tgid_lock(dev)

文件关闭时扫描全部 TFACC，释放当前线程或当前 TGID 持有的所有锁，防止异常退出留下永久锁。

tf_get_app_lock_records(p)

把 500 条历史复制给用户；仍持有的记录把 unlockTime 临时更新成当前 jiffies，便于用户计算当前持续时间。

tf_get_app_usage(pp)

生成数组：

- p[0]：全部 TFACC 近 15 秒总利用率；
- p[1]：全部 TFACC 近 60 秒总利用率；
- p[2]：全部 TFACC 近 300 秒总利用率；
- p[3+i]：TFACC i 近 15 秒利用率，千分比；
- 不存在的 TFACC 为 -1。

13.5 打开进程列表函数

push_pid()

在 pid_mutex 下把当前 PID/TGID 放入最多 64 项的数组；同一 PID 重复打开不会重复插入。

pop_pid()

移除当前 PID 的记录。当前主 release 路径没有调用它，而是调用 pop_tgid()。

pop_tgid()

移除当前 TGID 的所有线程记录。

get_pids(p)

顺序把所有有效 PID 写到用户数组，最后写 -1 作为终止标记。

init_pids()

把 64 项 PID/TGID 表初始化为 -1。

13.6 设备对象和字符设备函数

tf_create_and_init_device(device_id_counter)

1. kzalloc 一个 tf_device；
2. 初始化 spinlock、busy 状态、TFACC 所有者和 buffer 哈希表；
3. 对每个 chip 的 8 个 TFACC 执行 ioremap；
4. 对每个 TFACC 对应 cache 执行 ioremap；
5. 填充 reg_buf/cache_reg_buf，以后 tf_mmap() 使用。

`index = core + chip * 8` 把多 chip 展平成一个数组。

tf_remove_device_buf(dev)

删除 `buf_list` 中所有 `kbuf` 元数据。实际池 offset 由 `tf_reset_memory_owner()` 单独重置。

tf_remove_tgid_buf(dev, tgid)

只删除属于一个 TGID 的 `kbuf` 元数据，用于还有其他进程打开设备时的 close。

tf_reset_memory_owner(tgid, reset_all)

重置 reserved block 和 HugePage region 的 owner/offset。 `reset_all=true` 清全部，否则只清指定 TGID。

tf_create_and_init_cdev(dev, device_id)

- 动态申请 major/minor；
- `device_create()` 生成 `/dev/thinkforce0`；
- `cdev_init/cdev_add()` 绑定 `tf_device_ops`；
- 保存 `drvdata`；
- 失败时反向销毁 device 和设备号。

tf_remove_cdev(dev)

执行 `cdev_del()`、`device_destroy()` 和 `unregister_chrdev_region()`。

13.7 文件操作函数

tf_open(inode, filp)

- 由 `inode` 中的 `cdev` 找到 `tf_device`；
- 在内存锁和短 spinlock 下增加 `isBusy`；
- 写入 `filp->private_data`；
- 把 PID/TGID 加入打开者列表。

硬件初始化代码位于 `#if 0`，当前真正初始化发生在模块 `probe`，而不是每次 `open`。

tf_release(inode, filp)

1. 减少 `isBusy` 并判断是否最后一个打开者；
2. 重置当前 TGID 或全部内存池 owner/offset；
3. 删除对应 `kbuf` 元数据；
4. 释放当前 TGID 的全部 TFACC 锁和 PID 记录；
5. 最后一个打开者关闭时对 lite TFACC 做软件复位并重新 enable cache。

因为驱动按 close 回收整个 TGID 的大块，TFEngine 的辅助 fd 被设计成进程生命周期内保持打开，避免运行期间关闭某个辅助 fd 触发旧 release 语义。

tf_mmap(filp, vma)

完整逻辑见第 9 章。额外注意：

- 先按 mmap ID 解析目标；
- 拒绝映射长度超过对象长度；
- 寄存器映射为 noncached；
- 普通 buffer 可选择 writecombine；
- 最终用 CPU 物理地址而非 DMA 地址建立页表。

13.8 ACP 高位地址函数

tfacc_write_direct_address_high(buf, high_addr)

更新 0x10 的两个 8 位 direct 高位字段。只由 tfacc_full_acp() 在驱动初始化或完整 reset 时调用，运行时不能改。

tfacc_write_cache_address_high(buf, high_addr)

更新 0x90/0x94 中 cache 侧两个 8 位高位字段，保留其他位并置必要 enable bit；最后 wmb + readback。初始化和每次需要切换 4 GiB 窗口时都会调用。

tfacc_set_cache_address_high(base, high_addr)

检查高位不超过 0xff，临时映射对应 ACP 控制块，调用 cache 写入 helper 后解除映射。它不触碰 direct 0x10。

tfacc_full_acp(base, highAddr)

驱动启动/复位时配置一个 ACP 控制块：

- 设置 0x10 中的 protection 字段；
- 初始化其他 ACP/cache 地址配置寄存器；
- 把 direct 和 cache 高位都设成初始 reserved DDR 窗口。

运行时高位切换不能直接调用这个函数，因为它会改 direct 路径。

tf_ioctl_set_address_high(dev, arg)

1. 复制并验证请求；
2. chip_id = tfacc_id / 8；
3. pair_id = (tfacc_id % 8) / 2；
4. base = chipGap * chip_id + acp_bases[pair_id]；
5. 在 app_mutex 下验证 pair 两颗 TFACC 均属于当前 TGID；
6. 只调用 tfacc_set_cache_address_high()。

返回 `-EPERM` 通常表示 TFEngine 没有正确成对持锁； `-EINVAL` 通常表示版本、flags、TFACC ID 或高位范围错误。

13.9 cache、时钟和软件复位 helper

tfacc_full_clap(base) / tfacc_full_unclap(base)

设置或清除时钟控制窗口 `0x84` 的低两位，名称表示 full TFACC 的 clamp 控制。
当前主要调用已被注释，保留用于硬件初始化调试。

tfacc_swrst_one_cache(base)

把 cache 的 `0xe0/0xf8/0xfc` 清零，执行单 cache 软件复位准备。

tfacc_enable_one_cache(base)

向 `0x50` 写启动值，等待 `0x54 == 1`，然后设置 `0x04` bit6。它包含无超时轮询，硬件异常时可能一直卡在内核。

tfacc_enable_one_uncache(base)

配置 cache/uncache 地址 map，使相应范围按源码设定进入 uncache 行为。具体地址窗口位定义要对照硬件手册。

tfacc_full_enable_interleave(base)

设置 cache 控制 `0x04` bit12，启用 interleave。当前初始化末尾的调用被注释。

tfacc_lite_swrst(BASE)

写 Lite TFACC `REG_CHICKEN_BASE(0xff)` 触发软件复位/状态配置。

tfacc_full_swrst(BASE, BASE1)

同时操作一对 full TFACC，配置 reset mask、buffer pointer reset 和 chicken register，中间用 `dmb sy` 保证顺序。

tfacc_lite_enable_cache(BASE, CACHE_BASE)

首次初始化时 reset cache 和 Lite TFACC； `cacheInitCnt` 达到 `6 * chips` 后避免重复 reset；随后总是调用 `tfacc_enable_one_cache()`。

tfacc_full_enable_cache(BASE, BASE1, CACHE0_BASE, CACHE1_BASE, CLKBASE, CFGBASE)

初始化共享的一对 full TFACC 和两个 cache：

- 复位两个 cache；
- 设置 MAU/cache reset mask；
- 手工复位 buffer pointer；

- 释放 reset；
- 最后 enable 两个 cache。

源码中存在大量注释掉的硬件实验路径。阅读时先抓住“首次 reset，之后 enable”这一主线。

tfacc_full_enable(base)

打开 full TFACC 时钟和 PLL：unlock、首次 reset PLL、等待 lock status、打开时钟并解除 clamp。chipInitCnt == 2 * chips 后不再重复 PLL reset。

tfacc_full_checkrstcond()

读取/更新 ACP 0x34 复位状态标志；模块参数 needReset=1 可强制相关状态切换。

tfacc_full_disable(base)

clamp full TFACC，按 skipFullSwRst 决定是否切 reset，再关闭相关时钟位。

tfacc_lite_enable(base) / tfacc_lite_disable(base)

分别打开和关闭 Lite TFACC 的时钟、reset 与 clamp 位。

13.10 旧 ioctl 和内存分配函数

tf_ioctl_clear(dev)

在 tf_memory_mutex 下清除全部 kbuf，重置所有 reserved/HugePage owner 和 offset，并把动态 mmap ID 回到 1。

tf_ioctl_reset(dev)

执行完整硬件 reset，然后重新 enable full/lite、初始化 ACP 高位和所有 cache。当前函数只使用 chip 0 的固定寄存器地址，多 chip 场景需要谨慎。

tf_ioctl_check_version(dev, arg)

要求 SDK version 至少为 1840；成功返回历史 kernel version 20190605，失败返回期望版本。versionIsRight 的 mmap 强制检查目前已被注释，因此它主要用于旧 SDK 兼容提示。

tf_init_kbuf(dev, kbuf_p)

这是混合分配器核心：

1. 验证长度；
2. 优先遍历相同 chip 的 reserved block；
3. 同时判断当前 TGID 是否已有 reserved anchor；
4. reserved 无空间且已有 anchor 时遍历 HugePage；

5. 避免 DMA 低位为 0 和跨 4 GiB 边界；
6. 填充 backend、地址、offset、owner 和 mmap ID。

分配策略是 bump-only，不能释放中间洞。单个进程结束时整体复位 offset。

tf_rollback_kbuf(kbuf_p)

只有失败分配位于池尾时才能把 bump pointer 回退。reserved 检查

`block->offset == start + len`，HugePage 同理。若不是最后一次分配就不回退，避免覆盖后续已分配区域。

tf_create_and_init_kbuf(dev, len)

分配并清零 `struct kbuf`，设置长度，调用 `tf_init_kbuf()`；失败释放元数据。它只负责创建描述符，不做用户复制和哈希插入。

tf_ioctl_create(dev, arg)

1. 从用户复制旧 `tf_buf_io_param`；
2. 设置目标 chip `useDDR2`；
3. 在 `tf_memory_mutex` 下调用大块分配器；
4. 高地址大块在返回前执行完整 64 位 cache invalid；
5. 记录下一次 mmap 是否 `writcombine`；
6. 把 `kbuf` 插入哈希表；
7. 返回 DMA 地址和 mmap ID；
8. 任何失败都回滚哈希、池 offset 和对象。

tf_ioctl_get_app_infos(dev, p)

薄包装，调用 `get_pids()` 把打开驱动的 PID 列表返回用户。参数 `dev` 当前未使用。

tf_ioctl(filp, cmd, arg)

统一 ioctl dispatcher：

- 检查 magic 和 NR；
- 根据完整 `cmd` 调用具体 handler；
- handler 成功统一返回 0，失败返回负 `errno`。

新增 ABI 时需要同时：定义 UAPI、提高 `TF_MAX_NR`、添加 switch case、实现严格参数校验。

13.11 sysfs、拓扑和调试函数

show_kernel_version(dev, attr, buf)

按 `reserveSize` 返回 Normal/1M Face/3M Face 等旧产品模式文字。对应 sysfs 属性当前没有实际 `sysfs_create_file()`，因此通常不会被调用。

set_my_kernel(dev, attr, buf, len)

旧 sysfs 写回调，不修改任何状态，直接返回写入长度。

output_tfacc_id(base)

映射一个 TFACC 寄存器窗口，向 dmesg 打印 version 和 ID，用于 probe 时确认硬件。

readSocketInfo()

1. 读取 eFuse，判断硬件是否禁用；
2. 从配置寄存器判断 dual socket / dual die；
3. 设置 chips = 1/2/4 和对应 chipGap；
4. 按 ddrSize 为每个 chip 创建 256 MiB reserved block；
5. block 起点正好位于 4 GiB 边界时跳过前 1 MiB。

跳过低 32 位全 0 的地址，与 HugePage 分配器的规避策略一致，避免旧 ABI/哨兵值和边界处理出现歧义。

tfacc_cache_debug(BASE)

打印 cache 前 0x200 字节寄存器，仅在 TF_DEBUG 下真正输出，用于硬件调试。

tf_init_module() / tf_cleanup_module() / tf_remove()

分别是 probe、remove/module-exit 和设备对象清理入口，已在第 6 章展开。

14. tf_hugepage_register.c 全部函数说明

这是普通用户态 C 程序，不是内核代码。它通过 sysfs 管理 HugeTLB 池，再通过 ioctl 把页面交给驱动。

usage(stream)

输出 --size/--list/--clear/--chip/--node/--no-grow/--device 用法。

hugepage_pool_paths(numa_node, ...)

根据是否指定 NUMA node，生成全局或 node 级

nr_hugepages/free_hugepages sysfs 路径，并检查字符串是否截断。

read_count_file(path, value)

读取一个 sysfs 计数文件，严格处理解析和关闭错误。

write_count_file(path, value)

写 `nr_hugepages`，触发内核把普通内存转换成 1 GiB HugeTLB 页。需要 root，且运行已久的系统可能因物理碎片化无法满足。

ensure_hugepage_pool(required_pages, numa_node, grow_pool)

- 读取总页数和空闲页数；
- 空闲足够则直接成功；
- `--no-grow` 下不足就失败；
- 默认把总页数增加“所缺的空闲页数”；
- 再次读取并确认实际创建成功；
- 失败时建议使用启动参数 `hugepagesz=1G hugepages=N`。

parse_size(text, result)

解析裸数字、G/g 或 T/t，检查乘法溢出、非零且必须是 1 GiB 的整数倍。

parse_nonnegative_int(text, result)

严格解析 `chip/node`，拒绝负数、尾随字符和超过 `INT32_MAX` 的值。

create_hugetlb_memfd()

调用 `memfd_create`，指定 `MFD_HUGETLB | MFD_HUGE_1GB`，得到一个以 1 GiB HugeTLB 页为后端的匿名文件。

bind_mapping_to_node(address, length, node)

未指定 `node` 时不操作；否则通过 `mbind(MPOL_BIND | MPOL_MF_STRICT)` 把映射策略绑定到指定 NUMA node。

list_regions(device_fd)

从 index 0 循环调用 `TF_HUGEPAGE_QUERY`，遇到 `ENOENT` 结束，打印 region ID、chip、物理/DMA 地址、已用容量和 owner。

clear_regions(device_fd)

调用 `TF_HUGEPAGE_CLEAR`。遇到 `EBUSY` 时提示停止 NPU 应用；成功打印解除注册的页数。它解除驱动 pin，但不会自动缩小 `sysfs nr_hugepages`。

register_regions(device_fd, size, chip_id, numa_node)

1. 创建 HugeTLB memfd 并 `ftruncate` 到总容量；
2. 每次 `mmap` 一个 1 GiB 文件区间；
3. 应用 NUMA policy；
4. 调用 `TF_HUGEPAGE_REGISTER`；

5. 打印返回地址；
6. 用户映射立即 munmap，依靠驱动 pin 保持页面；
7. 中途失败时已成功注册的页仍然可用。

main(argc, argv)

- getopt_long 解析参数；
- 强制 --size/--list/--clear 三选一；
- 打开 /dev/thinkforce0；
- 注册路径先确保池容量，再逐页注册，最后自动列出；
- 用进程退出码表达成功/失败。

15. Makefile 和安装脚本

15.1 Makefile

- obj-m += tfacc2.o 告诉 Kbuild 生成 tfacc2.ko；
- KDIR=/lib/modules/\$(uname -r)/build 指向当前内核构建目录；
- make tfacc2 先编译用户工具，再构建、安装模块并执行 depmod；
- make tf_hugepage_register 只编译用户工具；
- make clean 清 Kbuild 产物和工具。

15.2 build_driver.sh

- 把 tfsmi/tfsmbios 安装到 /usr/local/bin；
- 创建隔离的 result[/IP] 构建目录；
- 复制驱动源码进去；
- 读取发行版名称到 LINUX_VERSION；
- 调用 make tfacc2 安装模块。

15.3 insmodTFDriver.sh

build_and_install_helper()

在源码目录编译 tf_hugepage_register，再以 0755 权限安装到 /usr/bin。

build_and_install_driver()

调用 build_driver.sh，定位 result 目录中生成的 HugePage 工具并安装到 /usr/bin。

主分支

- 默认 insmod：模块已加载时只更新工具；模块未加载时先尝试现有 modprobe，找不到已安装模块才从源码构建；

- `rm` : `modprobe -r tfacc2` ;
- `delete` : 删除 `/lib/modules` 下已安装的 `tfacc2.ko` 。

测试源码新版本时，需要确认不是重新加载了旧 `.ko` :

```
sudo ./insmodTFDriver.sh rm
sudo ./insmodTFDriver.sh delete
sudo ./insmodTFDriver.sh
cat /sys/module/tfacc2/version
```

16. TFEEngine 如何调用新驱动能力

这部分不是内核模块，但决定高地址方案能否正确使用。

16.1 MmapBuf

- `LowAddr()/HighAddr()` 从 64 位 NPU DMA 地址拆出低/高 32 位；
- 命令缓冲区额外保存 `commandAddressHigh` 软件元数据；
- 高地址 buffer 析构前调用 `DriverInvalidateCache()`，绕过旧 `tfnn` 的高位截断。

16.2 TFACCOpDescriptor::SetInOutPtr()

- 检查所有输入和输出位于同一 4 GiB 窗口；
- 检查高位不超过 8 bit；
- 硬件命令只写低位地址；
- 高位不再伪造成 NPU register command，而是保存在 `command buffer` 元数据里。

16.3 DriverRuntime.cpp

DriverFd()

进程内只打开一次 `/dev/thinkforce0` 并保持到进程退出。这样不会因为辅助 `ioctl fd` 提前 `close` 而触发驱动旧的按 `TGID release` 行为。

DriverInvalidateCache()

把 64 位 DMA 范围封装成 `TF_CACHE_INVALIDATE` `ioctl`。

DriverSetAddressHigh()

把运行 TFACC ID 和高位封装成 `TF_SET_ADDRESS_HIGH` `ioctl`；失败时打印 `errno`。

16.4 ProcessHighAddressLease

进程内每个 pair 有 {addressHigh, users, programmed} :

- `users == 0` : 首个算子取得状态, 负责调用驱动;
- 高位相同且 `programmed == true` : 另一个 TFACC 可以并发;
- 高位不同: 等待 `users` 降到 0;
- 首个线程写驱动前 `programmed == false` , 同高位线程也必须等待, 避免抢先启动;
- 同步 Forward 返回或异步 Wait 完成时 lease 析构、计数减一。

16.5 CoreInfo::LockPair()

进程内引用计数复用一对跨进程锁。首个用户通过稳定 tfnn ABI 分别锁两颗 TFACC; 最后一个用户才成对解锁。

16.6 NPU40THandler::Forward()/Launch()

执行顺序必须是:

```
选择/锁定 runtime pair
-> 获取进程内高位 lease
-> 首用户 ioctl 写 cache high
-> MarkProgrammed, 放行同高位线程
-> 启动 NPU
-> 硬件完成后释放 lease
```

不能把驱动写高位放到 NPU 命令流, 因为 `0xFE170000` 等是 CPU 物理 MMIO 控制块, 不是 NPU `TFACCRegisterBlasop` 能访问的内部寄存器编号。

17. 常用调试方法

17.1 确认模块和设备

```
lsmod | grep tfacc2
cat /sys/module/tfacc2/version
ls -l /dev/thinkforce0
dmesg | tail -100
```

17.2 查看 HugePage 池

```
for f in /sys/devices/system/node/node*/hugepages/hugepages-1048576kB/{nr_hugepages,free_hugepages}
do
    echo "$f: $(cat "$f")"
done

tf_hugepage_register --list
```

17.3 常见 errno

errno	名称	在本驱动中的常见原因
1	EPERM	无 CAP_SYS_ADMIN；高位 ioctl 未成对持锁；cache 范围不属于当前 TGID
12	ENOMEM	reserved/HugePage 空间不足；第一块 reserved anchor 失败；内核分配失败
14	EFAULT	用户指针不可访问；copy_to/from_user 失败；pin 页不完整
16	EBUSY	清理 HugePage 时仍有 NPU 应用或 region owner
17	EEXIST	HugePage 物理范围重复注册
22	EINVAL	UAPI 版本/flags/长度/chip/高位非法；可能仍加载旧驱动
34	ERANGE	内部高位 helper 的防御检查；当前 ioctl 会更早以 EINVAL 拒绝该输入
95	EOPNOTSUPP	没有可用 dma_device，通常是非 ACPI 路径
110	ETIMEDOUT	cache invalid ACK 超时

17.4 驱动日志

临时打开 tfacc2.h 中的 TF_DEBUG 可启用大量 DPRINTK。不要在高频执行路径长期打开，日志会显著影响时序和性能。

发生硬件 stall 后，优先保存：

- 应用打印的 command/result queue head/tail；
- dmesg 中的 cache invalid 超时和模块版本；
- 失败 ioctl 的 errno；
- tf_hugepage_register --list 输出；
- 当前算子的输入/输出 DMA 地址及高 32 位。

18. 当前代码中的历史风险与改进方向

这部分很重要：能在目标机器运行，不等于所有写法都是推荐的 Linux 驱动范式。

18.1 MMIO 资源生命周期不完整

`tf_create_and_init_device()` 对大量窗口 `ioremap()`，但失败路径和卸载路径没有完整逐项 `iounmap()`。建议集中保存映射数量，并在统一 `unwind/remove` 中反向解除。

更现代的 platform driver 可使用 `devm_ioremap_resource()` 自动绑定设备生命周期。

18.2 旧 MMIO 代码未检查 NULL

`cpu_write()`、多处 `enable/reset helper` 假设 `ioremap()` 必然成功，并使用普通 `volatile` 指针。建议改为：

- 检查映射结果；
- 使用 `readl/writel`；
- `helper` 返回 `errno`；
- 上层初始化失败时停止 `probe`，而不是继续访问。

18.3 spinlock 内执行慢操作

`tf_release()` 的最后用户分支持有 `dev->lock` 时调用 `ioremap`、`cache enable` 和无超时轮询。这些操作可能睡眠或耗时，不应放在 `spinlock` 临界区。

建议 `spinlock` 只保护 `isBusy` 和 `last_user` 判定，释放 `spinlock` 后再做硬件操作；如要防止并发 `open`，使用更高层 `mutex`/状态机。

18.4 无超时硬件轮询

`tfacc_enable_one_cache()` 和 `PLL lock` 等路径可能无限循环。硬件异常会让执行系统调用的任务永久卡在内核。建议统一使用 `readl_poll_timeout()` 或带截止时间的轮询，并返回 `-ETIMEDOUT`。

18.5 app lock 使用 busy wait

`tf_app_try_lock()` 每 10 微秒 `udelay`，等待 100 ms 就会消耗大量 CPU。更好的方案是 `waitqueue`：解锁时 `wake_up`，等待者可睡眠并支持超时。

18.6 用户复制返回值处理不完整

`tf_get_reserve_ddr_blocks()` 和 `tf_get_app_lock_records()` 忽略 `copy_to_user()` 返回值；`__get_user/__put_user` 也有多处未检查。用户地址错误时可能返回假成功。建议每次检查并立即返回 `-EFAULT`。

18.7 mmap 权限和所有权

动态 `buffer` 的 `mmap` 查找只按 `mmap_id`，没有验证 `owner_tgid`；寄存器窗口也可由打开设备的任意进程映射。生产安全模型应加入权限和所有权校验，必要时限制

CAP_SYS_RAWIO 。

18.8 全局状态较多

isNextUncache、versionIsRight、useDDR2 等是全局或 device 级状态，而不是每个 file descriptor 的上下文。多进程并发时可能相互影响。

建议定义 struct tf_file_context 放进 filp->private_data，其中保存调用进程的 chip、下一次 mmap 属性和资源引用；设备对象另存于 context。

18.9 release 的“按 TGID 全清”语义

同一进程打开多个 fd 时，关闭任意一个 fd 就会释放该 TGID 的锁和 buffer 元数据。当前 TFEngine 通过长期保持辅助 fd 避免触发，但更稳健的驱动应维护每 TGID 或每 file-context 引用计数，最后一个相关 fd 关闭才清理。

18.10 reserved/HugePage 分配器只能尾部回滚

它是 bump allocator，不能复用中间空洞。适合“加载模型时批量申请、进程结束整体释放”，不适合长生命周期中频繁申请/释放不同尺寸对象。需要更灵活时可使用区间树、gen_pool 或 bitmap allocator。

18.11 多 chip 路径需要专项审查

- tf_ioctl_reset() 只使用 chip 0 固定地址；
- tfacc_full_acp() 对特殊 ACP base 的相等判断没有消除 chipGap；
- 初始化高位公式包含 $0x80 / (\text{chips} / 2)$ ，假设实际机器不会是 $\text{chips} == 1$ ；
- close 路径重新 enable cache 时部分地址没有加 gap。

这些问题在当前硬件拓扑可能没有触发，但扩展到新板型前应逐项对照寄存器手册测试。

18.12 旧 ABI 的结构体定义不够稳健

部分旧 ioctl 用“结构体指针类型”参与 _IOWR，编码的是指针大小而不是结构体大小，且结构体含 bool/long。这会增加 32/64 位用户态兼容风险。

新 UAPI 应继续使用固定宽度整数、真实结构体类型、version/flags/reserved 字段，并在必要时提供 compat_ioctl。

19. 推荐阅读顺序和练习

第一遍：只看主干

1. tf_device_ops；

2. `tf_init_module()` ;
3. `tf_open()` ;
4. `tf_ioctl()` ;
5. `tf_ioctl_create()` -> `tf_init_kbuf()` ;
6. `tf_mmap()` ;
7. `tf_release()` ;
8. `tf_cleanup_module()` 。

第二遍：看大模型扩展

1. 三个新 UAPI;
2. `tf_hugepage_register` 工具;
3. `tf_ioctl_register_hugepage()` ;
4. HugePage fallback 分配;
5. `tf_ioctl_cache_invalidate()` ;
6. `tf_ioctl_set_address_high()` ;
7. TFEngine 的 pair lock 和 high-address lease。

第三遍：动手验证

建议先做只读观察，不直接改寄存器：

```
strace -e openat,ioctl,mmap,munmap tf_hugepage_register --list  
dmesg -w
```

然后在源码中给一个低频 ioctl 添加临时 `pr_info`，重新编译模块，观察一次用户调用如何进入内核。熟悉后再研究 `readl/writel` 和硬件初始化时序。

最重要的习惯是：每增加一个资源获取动作，都立即设计相应失败回滚和卸载释放；每增加一个用户输入，都先校验范围、所有权和整数溢出；每修改一个共享硬件寄存器，都先明确它影响的是一颗 core、一个 pair、一个 chip，还是整机。